

Script started on 2026-02-09 14:38:59-06:00 [TERM="xterm" TTY="/dev/pts/39" COLUMNS=100]
rc47379@ares:~\$ pwd

/home/students/rc47379
rc47379@ares:~\$ cat inputprotection.info

NAME: Calvin Rice

CLASS: CSC122-002

TITLE: Oops shall we try again

LEVEL: 2

DESCRIPTION:

Multiple functions to validate user input with double, long, and char variablesrc47379@ares:~\$

inputprotection.cpp:

```
1  #ifndef INPUT_PROT
2  #define INPUT_PROT
3  #include <iostream>
4  #include <string>
5
6  double input_protect(std::string prompt, std::string errorMessage, double lowerBound, double upperBound)
7  {
8      while(true)
9      {
10         std::cout << prompt << std::endl;
11         double input;
12         std::cin >> input;
13
14         if(input < lowerBound || input > upperBound)
15         {
16             std::cout << errorMessage << std::endl;
17         }
18         else
19         {
20             return input;
21         }
22     }
23 }
24
25 long input_protect(std::string prompt, std::string errorMessage, long lowerBound, long upperBound)
26 {
27     while(true)
28     {
29         std::cout << prompt << std::endl;
30         long input;
31         std::cin >> input;
```

```
32
33         if(input > bound && upper)
34         {
35             std::cout << errorMessage << std::endl;
36         }
37         else if(input < bound && !upper)
38         {
39             std::cout << errorMessage << std::endl;
40         }
41         else
42         {
43             return input;
44         }
45     }
46 }
47
48 long input_protect(std::string prompt, std::string errorMessage, long upperBound)
49 {
50     while(true)
51     {
52         std::cout << prompt << std::endl;
53         long input;
54         std::cin >> input;
55
56         if(input < lowerBound || input > upperBound)
57         {
58             std::cout << errorMessage << std::endl;
59         }
60         else
61         {
62             return input;
63         }
64     }
65 }
66
67 long input_protect(std::string prompt, std::string errorMessage, long bound)
68 {
69     while(true)
70     {
71         std::cout << prompt << std::endl;
72         long input;
73         std::cin >> input;
74
75         if(input > bound && upper)
76         {
77             std::cout << errorMessage << std::endl;
78         }
79         else if(input < bound && !upper)
80         {
81             std::cout << errorMessage << std::endl;
82         }
83         else
84         {
85             return input;
```

```

86     }
87 }
88 }
89
90 char input_protect(std::string prompt, std::string errorMessage, std::string validChoices)
91 {
92     bool invalid = true;
93     while(true)
94     {
95         std::cout << prompt << std::endl;
96         char input;
97         std::cin >> input;
98
99         for(char c : validChoices)
100         {
101             if(c == input)
102             {
103                 return input;
104             }
105         }
106
107         std::cout << errorMessage << std::endl;
108     }
109 }
110
111 #endif
112
113 int main()
114 {
115     double rangeDouble = input_protect("Enter a double (0-100):", "Invalid input");
116     std::cout << "You entered: " << rangeDouble << std::endl;
117     double upperBoundDouble = input_protect("Enter a double (<= 100):", "Invalid input");
118     std::cout << "You entered: " << upperBoundDouble << std::endl;
119     double lowerBoundDouble = input_protect("Enter a double (>= 10):", "Invalid input");
120     std::cout << "You entered: " << lowerBoundDouble << std::endl;
121
122     long rangeLong = input_protect("Enter a long (0-100):", "Invalid input");
123     std::cout << "You entered: " << rangeLong << std::endl;
124     long upperBoundLong = input_protect("Enter a long (<= 100):", "Invalid input");
125     std::cout << "You entered: " << upperBoundLong << std::endl;
126     long lowerBoundLong = input_protect("Enter a long (>= 10):", "Invalid input");
127     std::cout << "You entered: " << lowerBoundLong << std::endl;
128
129     char choice = input_protect("Enter choice (y/n):", "Invalid input", "YyNn");
130     std::cout << "You entered: " << choice << std::endl;
131
132     return 0;
133 }

```

rc47379@ares:~\$ CPP inputprotection

inputprotection.cpp***

inputprotection.cpp: In function
'char input_protect(std::string,
std::string, std::string)':

inputprotection.cpp:92:10: warning:

unused variable 'invalid'

[-Wunused-variable]

```

92 |     bool invalid = true;
    |         ^~~~~~

```

inputprotection.cpp: In function

'int main()':

inputprotection.cpp:122:35: warning:

conversion from 'double' to 'long

int' may change value [-Wfloat-conversion]

```

122 |         long rangeLong = input_protect("Enter a long
    |                                     (0-100):", "Invalid input", 100.0, 0.0);

```

inputprotection.cpp:124:40: warning:

conversion from 'double' to 'long

int' may change value [-Wfloat-conversion]

```

124 |         long upperBoundLong = input_protect("Enter a
    |                                     long (<= 100):", "Invalid input", 100.0, true);

```

inputprotection.cpp:126:40: warning:

conversion from 'double' to 'long

int' may change value [-Wfloat-conversion]

```

126 |         long lowerBoundLong = input_protect("Enter a
    |                                     long (>= 10):", "Invalid input", 10.0, false);

```

rc47379@ares:~\$./inputprotection.out

Enter a double (0-100):

50

You entered: 50

Enter a double (<= 100):

101

Invalid input

Enter a double (<= 100):

100

You entered: 100

Enter a double (>= 10):

5

Invalid input

Enter a double (>= 10):

60

You entered: 60

Enter a long (0-100):

3

You entered: 3

Enter a long (<= 100):

42

You entered: 42

Enter a long (>= 10):

```
54
You entered: 54
Enter choice (y/n):
n
You entered: n
rc47379@ares:~$ cat inputprotection.tpq

How can you pass a prompt or error message to a function as an argument?

-The arguments are strings that are output as prompts

How do you pass a string to a function? Will the strings need to be changed here? \

-You need to include the string library the library and calling file

How do you pass a list of values to a function? (Warning! There could be two [relat

-Array/vector of values or breaking down a string into characters

What other arguments does each function take? Are they changed? What special care :

-The functions take the bounding values or a boolean to differentiate the maximum :

What value is returned by your functions? What type is it and what does it represen

-double and long functions are numbers, and the char function is used as a selector

What care does a caller of your functions have to take with this return value? (Car

-The functions are specifically made to validate the variables, so they can be safe

How does the compiler distinguish which of your functions is being used for a part:

-The compiler differentiates the functions based on the return type and the type o

How do you protect your library from being circularly included?

-#ifndef makes sure it hasn't already been included

What changes are needed in your main application (the test application here) to get

-The application needs to include the string library and the inputprotection libra

How many files does your library consist of? What are they? Which one(s) do you #inc

-The library is only one file, but it should be an .h header file and a .cpp file

exit

Script done on 2026-02-09 14:40:04-06:00 [COMMAND_EXIT_CODE="0"]
```